MuSTNet: SAT-based Exact Multi-Stage Transistor Network Synthesis with Placement Awareness

Jiun-Cheng Tsai[1], Wei-Min Hsu[1], Kuei-Lin Wu[1], Hsuan-Ming Huang[1], Jen-Hang Yang[1], Heng-Liang Huang[1], Yen-Ju Su[2], Charles H. -P. Wen[2]
[1]MediaTek Inc., Hsinchu, Taiwan,
[2]National Yang Ming Chiao Tung University, Hsinchu, Taiwan
email: jiun-cheng.tsai@mediatek.com

2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD) | 979-8-3315-1560-7/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICCAD66269.2025.11240651

***Abstract*—Optimizing power, performance, and area in IC designs remains a key focus. However, the limited functionality of standard cell libraries restricts further optimization. A promising solution involves developing customized complex gates that integrate multiple basic gate functions into a single gate at the transistor level. While prior research has extensively explored transistor network optimization of the complex gates, most studies still focus on 1-stage networks, limiting the potential for deeper optimization. Furthermore, existing methodologies often neglect considering transistor placement during network synthesis, potentially leading to suboptimal area even if the transistor count is reduced. To address these limitations, we propose MuSTNet, a SAT-based exact synthesis framework that minimizing transistor networks by incorporating two key innovations: (1) *multi-stage hierarchy* for deeper optimization, and (2) *transistor placement* constraints to simultaneously minimize both transistor count and physical area. Experimental results demonstrate that MuSTNet surpasses previous studies, achieving an 18% reduction in transistor count for 4-input P-class functions and a 12.9% reduction for multi-output functions. When applied to an industrial library benchmark, MuSTNet yields a 6.3% area reduction compared to the approach neglecting placement constraints. Moreover, MuSTNet has been applied to complex gate generation, allowing simultaneous functional and topological optimization at the transistor level. Compared to traditional cell-level design, this approach reduces transistor count by 12.3% and area by 21.4%, demonstrating its potential in advanced IC design.**

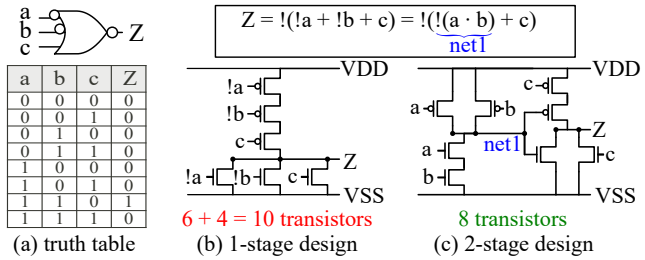
***Index Terms*—transistor network, placement-aware, exact synthesis, SAT.**

I. Introduction

In VLSI digital design, optimizing critical parameters such as area, power, and delay remains a central challenge. While traditional standard cell libraries are widely used, their limited functionality often restricts deeper optimization. A promising solution lies in customized complex gates (known as supergates) [1]–[15], which integrate multiple basic gates functions into a single gate at the transistor-level, enabling finer optimization of both transistor networks and physical layouts. Consequently, extensive research has been proposed for the optimization of transistor networks, which can be broadly categorized into three methodologies: factorization-based [1], [2], graph-based [4]–[7], and SAT-based [10], [13] approaches.

Factorization-based approaches simplify Boolean function by transforming it into minimal factored forms, then constructing series-parallel (SP) transistor networks based on the optimized expressions (the literals and the and/or operations). However, this approach is inherently constrained on SP structures limits their ability to exploit non-series-parallel (NSP) structure, potentially missing opportunities for further transistor reduction.

The NSP networks, also referred to as bridge network, enables a *bridge* between two or more intermediate nodes in the series connections, enabling transistor count reduction through path-sharing. To take the advantages, several graph-based techniques [4]–[7] have been developed, focusing on optimizing path-sharing to minimize transistor count. For instance, prior work [6] employs kernel-finding and complex-sharing mechanisms to significantly reduce transistor counts in single-output complex gates. Further advancements, such as MOTO-X [7], gradually optimize bridge of the transistor network, extending applicability to multi-output functions.



a	b	c	Z
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

(a) truth table

Fig. 1. Example of NOR3 cell with two inverted inputs: (a) truth table, (b) 1-stage design with 10 transistors, (c) 2-stage design with only 8 transistors.

Inspired by the success of SAT-based exact synthesis at the logic level [16], [17], recent work has introduced MiniTNtk [10], a transistor-level exact synthesis framework minimizing 1-stage transistor network on single-output design. This approach model the transistor network synthesis problem as a SAT problem, implicitly exploring both SP and NSP structures. By leveraging SAT solvers and exact synthesis approaches, optimality can be guaranteed, allowing for a further reduction in transistor count compared to graph-based methods. An extension work [13] proposed a heuristic-guided exact algorithm to accommodate designs from single output to multi-output. Although the SAT-based approach demonstrates optimality in transistor network synthesis, a critical question remains: *Is the transistor network truly minimized if we only consider the 1-stage hierarchy?*

Fig. 1 addresses this question. Consider a NOR3 gate with two inverted inputs where the symbol and truth table as illustrated in Fig. 1(a). Previous approaches for this function typically produce a 1-stage transistor network, requiring 10 transistors including 2 inverters for the inputs, a and b as shown in Fig. 1(b). However, by decomposing the function carefully, a more optimal solution can be achieved using a 2-stage hierarchy, as depicted in Fig. 1(c). In the first stage, $net1$ is generated as $!(a \cdot b)$, and output Z is then produced using c and $net1$ in the second stage. This 2-stage design results in a total of only 8 transistors, achieving a 20% reduction compared to 1-stage design. This example demonstrates multi-stage design as a crucial strategy that must be integrated into transistor network synthesis for deeper optimization.

Moreover, in optimizing complex cells, one of our primary goals is to reduce area by minimizing the transistor network. A critical question arises: *do networks with the fewest transistors inherently yield the*

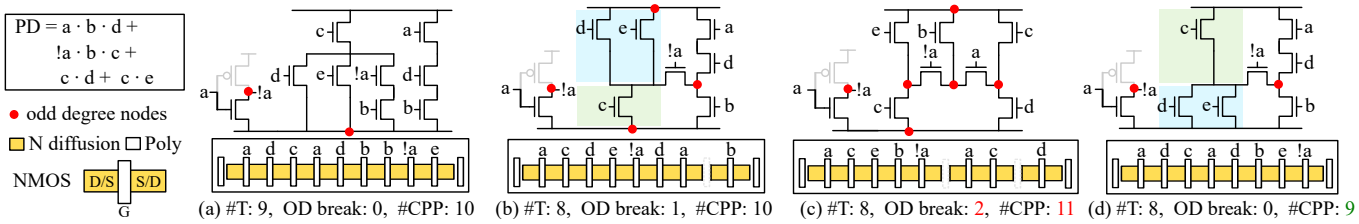


Fig. 2. Example of PD network implementation with its placement (a) 9 transistors with no OD break. (b) 8 transistors with 1 OD break. (c) 8 transistors but with 2 OD break. (d) 8 transistors with no OD break.

smallest area? The answer depends on the physical implementation of the network. Specifically, during transistor placement stage, the adjacent transistors must share the same signal in their common diffusion regions; otherwise, a diffusion (OD) break is required. Consequently, even a transistor network with a low transistor count may result in significant area overhead if its arrangement necessitates numerous OD breaks. To illustrate this, Fig. 2 presents an analysis of the pull-down (PD) network, where OD breaks are calculated using the Euler path technique [18]. For a network with n odd-degree nodes (indicated by red points), the required OD breaks are $(n–2)/2$ where $n > 0$. In Fig. 2, the corresponding N-type transistor placement is shown beneath each network, with the area represented by number of contacted poly pitch (#CPP). Fig. 2(a) depicts a SP implementation of this function using 9 transistors (#T), achieving 0 OD breaks and a #CPP of 10. Conversely, Fig. 2(b) shows a NSP network implementation reducing transistors to 8, but requiring 1 OD break, resulting in the same area as Fig. 2(a). An interesting case is presented in Fig. 2(c), where an alternative NSP network implementation with 8 transistors but 2 OD breaks, leading to 1 additional #CPP compared to Fig. 2(a). Finally, the optimal solution could be illustrated in Fig. 2(d), maintaining the same hierarchy as Fig. 2(b) but reordering the transistor stack (marked as blue and green region). This results in 8 transistors without OD breaks, successfully reducing the #CPP from 10 to 9 (a 10% reduction). This example demonstrates the necessity of integrating transistor placement considerations during minimizing transistor network to preserve area efficiency.

Therefore, to address these limitations in transistor network generation, we propose MuSTNet, a SAT-based exact multi-stage transistor networks synthesis framework with transistor placement awareness on CMOS technology. MuSTNet minimizes the transistor count by leveraging multi-stage design while incorporating transistor placement constraints to preserve area efficiency. The main contributions of this work are:

1) SAT-based multi-stage transistor network synthesis: To the best of our knowledge, this is the first work to formulate the multi-stage transistor network synthesis problem as a SAT problem. Crucially, we carefully permit internal nodes to serve as transistor gates, enabling multi-stage hierarchical networks rather than restricting solutions to 1-stage design. This facilitates deeper optimization, with experimental results showing that MuSTNet outperforms prior SAT-based approaches, achieving an 18% reduction in transistor count for 4-input P-class functions and a 12.9% reduction for multi-output functions.
2) Placement-aware transistor network minimization: MuSTNet incorporates transistor placement constraints to ensure area-efficient physical implementation. When evaluated on an industrial library benchmark, our framework delivers a 6.3% area reduction compared to the approach ignoring placement considerations.
3) Transistor-level optimization for complex gates: MuSTNet enables simultaneous functional and topological optimization at the transistor level. Compared to traditional cell-level design, this approach reduces transistor count by 12.3% and area by 21.4%, demonstrating its promise for advanced IC design.

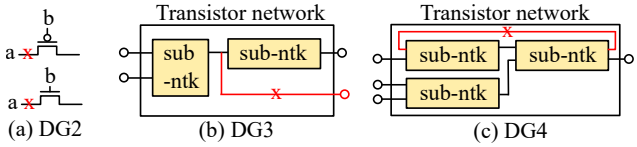


Fig. 3. Illustration of design guideline: (a) DG2: Prohibit primary inputs from driving source/drain terminals. (b) DG3: Never use primary outputs as gate terminals for internal transistors. (c) DG4: Eliminate self-driving loops.

II. Preliminary

A. CMOS Transistor Network Design

In this work, we focus on *static CMOS technology*, which is widely adopted in modern digital design. A CMOS circuit consists of two key components: a pull-up (PU) network and a pull-down (PD) network, which connect the primary output to V_{DD} or V_{SS} to set the logic to '1' or '0', respectively. Following established design methodologies [19], we highlight four critical design guidelines (***DG***) for ensuring reliability and robustness of static CMOS complex gates.

DG1: Ensure one and only one strong '1' or '0' to drive the gate terminal. Our multi-stage hierarchy allows internal nodes to serve as gate terminals for other transistors. However, not all internal nodes are viable candidates. To guarantee stability, these nodes must connect to either V_{DD} (strong '1') or V_{SS} (strong '0') through a low-resistance path under all input patterns. Effectively, they should behave like primary outputs, delivering a static and strong logic level ('1' or '0').

DG2: Prohibit primary inputs from driving source/drain terminals. Unlike Pass-Transistor Logic (PTL) design [19], allowing primary inputs to drive both gate and source/drain terminals, our methodology avoids this approach (Fig. 3(a)). Although PTL can reduce transistor count, it introduces a voltage drop due to threshold voltage, weakening signal integrity, especially in cascaded stages. This degradation compromises robustness, thereby rendering the design unreliable.

DG3: Never use primary outputs as gate terminals for internal transistors. Using a primary output to drive the gate of an internal transistor leads to instability or unintended behavior (Fig. 3(b)). External factors like unpredictable loading, noise, or signal degradation may lead to improper biasing and delayed switching. Notably, noise coupling may induce false switching in the driven transistor, leading to functional failures.

DG4: Eliminate self-driving loops. A loop structure (Fig. 3(c)) occurs when a net feeds back to its own transistor input after traversing one or more sub-networks. Such loops lack an initial value

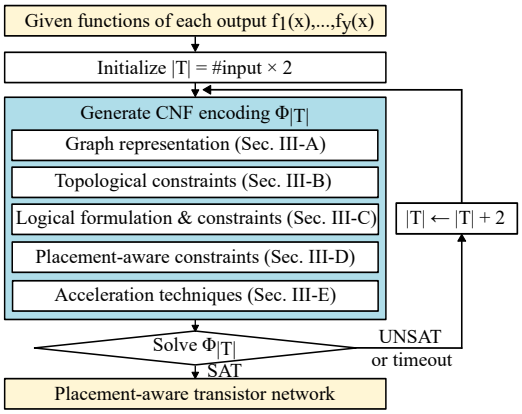


Fig. 4. Overall framework of MuSTNet

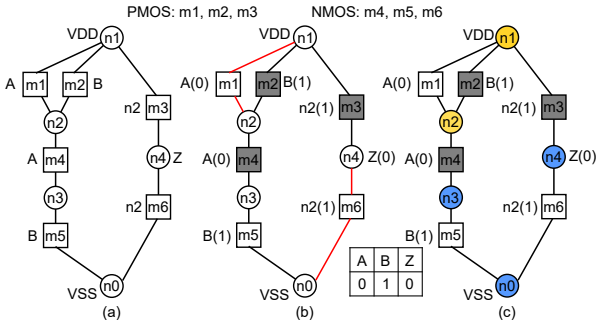


Fig. 5. Example graph of 2-input AND (a) graph representation (b) flow diagram (c) connected component diagram

and may cause unstable condition for the entire transistor network, thereby not allowed in our design.

III. The Framework of MuSTNet

Within our SAT-based exact synthesis framework, MuSTNet, we investigate a fundamental question of *whether a transistor network with x inputs and y outputs can implement the target Boolean functions $f_1, ..., f_y$ using $|T|$ transistors.*

Figure 4 depicted the overall framework where begins by accepting the target Boolean functions $f_1(x), ..., f_y(x)$ as input. Since each primary input must be used at least once, we initialize the transistor count $|T|$ to twice the number of primary inputs, accounting for both PMOS and NMOS transistors. Using the given functions $f_1(x), ..., f_y(x)$ and $|T|$, we encode the SAT formula $\Phi_{|T|}$ based on a set of constraints. A SAT solver then determines whether a feasible transistor network exists. If $\Phi_{|T|}$ is SAT, the corresponding network is synthesized from the satisfying assignment. Conversely, if $\Phi_{|T|}$ is UNSAT or timeout, we increase $|T|$ by 2 (adding one PMOS and one NMOS transistor) and repeat the process until a valid solution is obtained. The following sections detail the formulations and constraints individually. Table I summarizes the notation used in this work.

A. Graph Representation of Transistor Network

In this work, we model the transistor network as a graph $G = (V, E)$, where the node set V comprises MOS nodes m and net nodes n, and the edge set E defines the connections between them (m and n). Each MOS node possesses an attribute specifying its gate signal. The number of net node is defined under the worst-case condition as $|T| + 1$, where all transistors are connected in series. Moreover, according to ***DG2***, the set of net node (N) does not contain input pin nets, which are forbidden to be source or drain of MOS. Note that V_{SS} and V_{DD} are pre-defined as n_0 and n_1, respectively. Once the edges and gate assignments are determined, a complete transistor network is formed. Unlike prior SAT-based approaches [10], [13], which treat the PU and PD networks as disjoint structures and synthesize them independently, our framework supports multi-stage designs where internal nodes may serve as transistor gates. Consequently, we must simultaneously solve for both the PU and PD networks to ensure internal nodes assume deterministic logic values, adhering to ***DG1***. Fig. 5(a) presents an exemplary graph for a 2-input AND gate where circles represent net nodes, squares denote MOS nodes, and the corresponding gate signal are labeled beside the MOS nodes.

TABLE I
Notation Table

Term	Description
$\|T\|$	number of transistors
S_{in}	set of signals of input pins
N	set of net nodes
N_{OUT}	set of net nodes of output pins
n_i	i-th net
m_i	i-th mos, pmos: $1 <= i <= \|T\|/2$, nmos: $\|T\|/2 + 1 <= i <= \|T\|$
p_k	k-th input pattern
e_{n_i,m_j}	0,1 if edge exists between n_i and m_j
g_{m_i,n_j}	0,1 if m_i has gate with internal net n_j
$l^{0/1}_{p_k,n_i}$	0,1 if logic of n_i equals 0/1 on pattern p_k
$f^{0/1}_{p_k,n_s,n_i,m_j}$	0,1 if exists flow from 0(V_{SS})/1(V_{DD}) to sink n_s on pattern p_k through edge (n_i, m_j)
a_{p_k,m_i}	0,1 if mos m_i is switched on under pattern p_k
$c^{0/1}_{p_k,n_i}$	0,1 if net node n_i is connected to 0(V_{SS})/1(V_{DD}) on p_k
$Tp^{P/N}_{m_i,j,f}$	0,1 if P/N mos m_i placed at column j with flip type f
$D/S_{m_i,n_j}$	0,1 if n_j is drain(D)/source(S) of mos m_i

B. Topological Constraints

To ensure a valid transistor network, some topological constraints must be obtained.

- *MOS valid constraint:* Each MOS node must have exactly one gate signal and exactly two edges, source and drain. In accordance with ***DG3***, the output pins N_{OUT} are omitted from the gate terminal.

$$\bigwedge_{i=1}^{|T|} \left(\sum_{In_j \in S_{in}} g_{m_i,In_j} + \sum_{n_j \in N \setminus N_{OUT}} g_{m_i,n_j} \right) = 1 \quad (1)$$

$$\bigwedge_{j=1}^{|T|} \left(\sum_{n_i \in N} e_{n_i,m_j} = 2 \right) \quad (2)$$

- *No floating node constraint:* Each node should not connect to only one MOS, except $V_{DD}(n_1)$ and $V_{SS}(n_0)$.

$$\bigwedge_{n_i \in N \setminus \{n_0,n_1\}} \left(\sum_{j=1}^{|T|} e_{n_i,m_j} \neq 1 \right) \quad (3)$$

- PMOS should not connect to V_{SS}, and NMOS should not connect to V_{DD}.

$$\bigwedge_{m_i \in pmos} \neg e_{n_0,m_i}, \quad \bigwedge_{m_i \in nmos} \neg e_{n_1,m_i} \quad (4)$$

- *Avoid always-on PMOS/NMOS*: Never tie a PMOS gate to V_{SS} or NMOS gate to V_{DD}. Note that connecting a PMOS gate to V_{DD}

(or NMOS gate to V_{SS}) is allowed, as it results in a non-functional dummy device.

$$\bigwedge_{m_i \in pmos} \neg g_{m_i,n_0}, \bigwedge_{m_i \in nmos} \neg g_{m_i,n_1} \tag{5}$$

C. Logical Formulation and Constraints

In this work, a logic value $L(n_i) \in \{0, 1, X\}$ of a net node n_i is encoded using a two-bit representation, $l^0, l^1 \in \{0, 1\}$, as defined by the following mapping:

l^0	l^1	L
0	1	1
1	0	0
0	0	X
1	1	Forbid

Here, $l^1 = 1$ (or $l^0 = 1$) for a net node n_i indicates that n_i is connected to V_{DD} (V_{SS}), meaning there exists a path between V_{DD} (V_{SS}) and n_i. Conversely, $l^1 = 0$ (or $l^0 = 0$) implies no such connection. However, when the both l^1 and l^0 are 1, it indicates a short path between V_{DD} and V_{SS}, which must be forbidden. To prevent this, we enforce the following constraint:

$$\bigwedge_{n_i \in N} \left(\neg l^0_{p_k,n_i} \wedge \neg l^1_{p_k,n_i}\right) \tag{6}$$

Our objective is to ensure the transistor network correctly implements the desired Boolean function across all input patterns p_k, where k ranges from 0 to $2^{|S_{in}|} - 1$. To achieve this, we assign logic values to output pins under each pattern p_k and verify if the network satisfies the assignment:

$$\bigwedge_{n_i \in N_{OUT}} \begin{cases} l^0_{p_k,n_i} = 1 & \text{, if } L(n_i) = 0 | p_k \\ l^1_{p_k,n_i} = 1 & \text{, if } L(n_i) = 1 | p_k \end{cases} \tag{7}$$

Furthermore, it is essential to ascertain not only the logical state of the output node but also that of any internal node functioning as a gate. Based on the ***DG1***, the logical value must assume either a strong 1 or 0, as formalized below:

$$\bigwedge_{i=1}^{|T|} \bigwedge_{n_j \in N \setminus N_{OUT}} \left(g_{m_i,n_j} \implies (l^0_{p_k,n_j} \vee l^1_{p_k,n_j})\right) \tag{8}$$

The terms l^1 and l^0 in Eq. 7 and 8 can be determined as follows. First, as previously established, for l^1 (or l^0) to be true at node n, it must be demonstrated that there exists at least one path between V_{DD} (or V_{SS}) and node n. This pathfinding problem can be modeled as a multi-commodity flow problem [20], [21], where V_{DD} and V_{SS} act as *source* nodes, and the remaining net nodes serve as *sink* nodes. For each source-sink pair, a flow variable is introduced to indicate whether a flow exists from the source to the sink node n through edge e under pattern p_k. This confirms the existence of a path, and the corresponding rules are defined as follows:

- *Edge existence constraints*: if a flow exists between n_i and m_j, it implies the presence of an edge.

$$\bigwedge_{l=0,1} \bigwedge_{n_s \in N} \bigwedge_{n_i \in N} \bigwedge_{j=1}^{|T|} f^l_{p_k,n_s,n_i,m_j} \implies e_{n_i,m_j} \tag{9}$$

- *Flow constraints for net node*: By definition, a 1-flow (or 0-flow) must originate only from V_{DD} (or V_{SS}), and no flow is permitted at the output node if it is not the sink node. For source-sink pairs exempt from verification, the net node may exhibit zero flow. Conversely, for paths requiring verification, source and sink nodes must have exactly one flow and other net nodes must either exhibit no flow (0), or balanced flow (2), ensuring that inflow equals outflow of the net node.

$$\bigwedge_{l=0,1} \bigwedge_{n_s \in N} \bigwedge_{n_i \in N} \sum_{j=1}^{|T|} f^l_{p_k,n_s,n_i,m_j} \begin{cases} = 0 & \text{if } (l = 1 \wedge i = 0) \textit{ or} (l = 0 \wedge i = 1) \textit{ or} \\ & (n_s \in N_{OUT} \wedge s \neq i), \\ \leq 1 & \text{if } (l = 1 \wedge i = 1) \textit{ or} (l = 0 \wedge i = 0) \textit{ or} \\ & (s = i), \\ = 2 \text{ or } 0 & \text{otherwise} \end{cases} \tag{10}$$

- *Flow constraints for MOS node*: Similar with intermediate net nodes, each MOS node must have either 2 or 0 flow.

$$\bigwedge_{l=0,1} \bigwedge_{n_s \in N} \bigwedge_{j=1}^{|T|} \sum_{n_i \in N} f^l_{p_k,n_s,n_i,m_j} = 2 \textit{ or } 0 \tag{11}$$

A flow can only pass through a MOS if it is switched on under the given pattern. Specifically, PMOS is on when gate signal is 0 and NMOS is on when gate signal is 1.

$$\bigwedge_{l=0,1} \bigwedge_{n_s \in N} \bigwedge_{n_i \in N} \bigwedge_{j=1}^{|T|} f^l_{p_k,n_s,n_i,m_j} \implies a_{p_k,m_j} \tag{12}$$

$$\bigwedge_{m_i \in pmos} a_{p_k,m_i} = \bigvee_{\forall In_j = 0 | p_k} g_{m_i,In_j} \vee \bigvee_{n_j \in N \setminus N_{OUT}} (g_{m_i,n_j} \wedge l^0_{p_k,n_j}) \tag{13}$$

$$\bigwedge_{m_i \in nmos} a_{p_k,m_i} = \bigvee_{\forall In_j = 1 | p_k} g_{m_i,In_j} \vee \bigvee_{n_j \in N \setminus N_{OUT}} (g_{m_i,n_j} \wedge l^1_{p_k,n_j}) \tag{14}$$

- *Flow implication constraints*: When a logic value is determined, the corresponding flow must exist, implying the presence of both a source and a sink. Below, we formulate the case for a 0-logic value on node n_i, which enforces a 0-flow between V_{SS} and node n_i. The formulation related to a 1-logic value can be derived in a similar manner.
- a 0-flow (f^0) from source $V_{SS}(n_0)$,

$$\bigwedge_{n_i \in N} l^0_{p_k,n_i} \implies \bigvee_{j=1}^{|T|} f^0_{p_k,n_i,n_0,m_j} \tag{15}$$

- a 0-flow to sink n_i,

$$\bigwedge_{n_i \in N} l^0_{p_k,n_i} \implies \bigvee_{j=1}^{|T|} f^0_{p_k,n_i,n_i,m_j} \tag{16}$$

Example 1: *Figure 5(b) shows an example of path connection for an AND2 gate under pattern A=0, B=1, Z=0. When Z=0, a path forms from V_{SS} to node $n4$, highlighted by the red line in the figure. Following the previous formulations, an internal node $n2$ will be assigned to logic 1 for activating the NMOS $m6$, which brings up another path from V_{DD} to node $n2$.*

Equations 9 to 16 outline the method for verifying that l^0 and l^1 are true at a target node. However, we must also ensure that no conflicting logic flow (e.g., 0-logic from V_{DD}) reaches the node. This requires that the node and the conflicting logic source reside in separate connected components of the graph. To identify connected components, we label two net nodes as belonging to the same component if they are connected through an *'on'* MOS node. The overall formulation is shown below:

- V_{DD} and V_{SS} component assignment:

$$c^1_{p_k,n_1} = 1, c^0_{p_k,n_0} = 1 \tag{17}$$

- *Connected component propagation*: Two nodes n_i, n_j are in the same connected component if they are linked via an *'on'* MOS m_n.

$$\bigwedge_{n=1}^{|T|} \bigwedge_{\substack{n_i,n_j \in N \\ i \neq j}} \{(e_{n_i,m_n} \wedge e_{n_j,m_n} \wedge a_{p_k,m_n}) \implies (c^1_{p_k,n_i} = c^1_{p_k,n_j}) \wedge (c^0_{p_k,n_i} = c^0_{p_k,n_j})\} \tag{18}$$

- Component isolation by logic value

$$\bigwedge_{n_i \in N} l^0_{p_k,n_i} \implies \neg c^1_{p_k,n_i}, \bigwedge_{n_i \in N} l^1_{p_k,n_i} \implies \neg c^0_{p_k,n_i} \tag{19}$$

Example 2: *Figure 5(c) illustrates the connected components for an AND2. Here, net nodes connected to V_{DD} are highlighted in yellow, while those connected to V_{SS} appear in blue. For instance, when node $n4$ is 0-logic, isolation from V_{DD} (yellow) is necessary to ensure functional correctness.*

Finally, to comply with ***DG4***, we introduce a constraint that eliminates loop structures by ensuring no cycles exist in the graph. This is achieved by preventing node revisits during traversal. Specifically, we impose a driving order among nodes, where gate net can only drive nets with a larger ID, thereby preventing any cyclic dependencies.

- *Self-driving elimination constraints*: if the gate of mos m_i is n_j, all flow to sink n_s should be 0 when $s < j$.

$$\bigwedge_{i=1}^{|T|} \bigwedge_{n_j \in N \setminus N_{out}} g_{m_i,n_j} \implies \bigwedge_{l=0,1} \bigwedge_{s=0}^{j-1} \bigwedge_{n_t \in N} \neg f^l_{p_k,n_s,n_t,m_i} \tag{20}$$

D. Placement-aware Constraints

To optimize the area efficiency of the generated transistor network, we introduce the following placement-aware constraints. For brevity, we formulate the constraints for PMOS, while the NMOS constraints can be derived analogously. Given a set of positions, (col), each PMOS m_i is assigned to a column j with flip-type $f \in \{0, 1\}$ (where 0: D-G-S, 1: S-G-D), denoted as $Tp^P_{m_i,j,f}$.

- *Transistor allocation constraint*: each MOS should be used exactly one time with only one flip type.

$$\bigwedge_{m_i \in pmos} \left(\sum_{j=1}^{col} \sum_{f=0,1} Tp^P_{m_i,j,f} = 1 \right) \tag{21}$$

- *Column allocation constraint*: each column should contain exactly one transistor.

$$\bigwedge_{j=1}^{col} \left(\sum_{m_i \in pmos} \sum_{f=0,1} Tp^P_{m_i,j,f} = 1 \right) \tag{22}$$

- *Diffusion net assignment*: During SAT solving, the drain and source of each transistor are determined dynamically. To ensure consistency, we assign the net with the smaller ID as drain and the net with the larger ID as source.

$$\bigwedge_{n=1}^{|T|} \bigwedge_{n_i \in N} \left(D_{m_n,n_i} = (e_{n_i,m_n} \wedge \neg \bigvee_{n_j \in N, j<i} e_{n_j,m_n}) \right) \tag{23}$$

$$\bigwedge_{n=1}^{|T|} \bigwedge_{n_i \in N} \left(S_{m_n,n_i} = (e_{n_i,m_n} \wedge \neg \bigvee_{n_j \in N, j>i} e_{n_j,m_n}) \right) \tag{24}$$

- *Diffusion sharing constraint*: Adjacent transistors must share a common diffusion net.

$$\bigwedge_{k=1}^{col-1} \bigwedge_{\substack{m_i,m_j \in pmos \\ i \neq j}} Tp^P_{m_i,k,f_0} \wedge Tp^P_{m_j,k+1,f_1} \implies \begin{cases} \bigwedge_{n_n \in N} (S_{m_i,n_n} = D_{m_j,n_n}) & \text{if } f_0 = 0, f_1 = 0 \\ \bigwedge_{n_n \in N} (S_{m_i,n_n} = S_{m_j,n_n}) & \text{if } f_0 = 0, f_1 = 1 \\ \bigwedge_{n_n \in N} (D_{m_i,n_n} = D_{m_j,n_n}) & \text{if } f_0 = 1, f_1 = 0 \\ \bigwedge_{n_n \in N} (D_{m_i,n_n} = S_{m_j,n_n}) & \text{if } f_0 = 1, f_1 = 1 \end{cases} \tag{25}$$

E. Acceleration Techniques

This section introduces two acceleration techniques to enhance the SAT solving process. Although each node in the graph is unique, the topology of a transistor network remains identical when the connectivity is the same, regardless of nodes. To prevent redundant exploration of equivalent topologies—where the SAT solver wastes time swapping nodes with identical connectivity—we impose an ordering constraint on net nodes and MOS nodes to expedite the computations.

- *Net node ordering*: A node n_i can only be used if its predecessor n_{i-1} has already been utilized.

$$\bigwedge_{n_i \in N} \left(\bigvee_{j=1}^{|T|} e_{n_i,m_j} \implies \bigvee_{j=1}^{|T|} e_{n_{i-1},m_j} \right) \tag{26}$$

- *MOS node ordering*: The ordering of MOS is determined based on connectivity through nodes. Specifically, each MOS must connect to V_{DD}/V_{SS}, output node, its predecessor via an already-utilized node or attach to at most two unassigned nodes. To enforce this, we define a candidate node set C_{m_j} for each MOS (m_j):

$$C_{m_j} = \{N_{OUT}, n_0, n_1, n_2, ..., n_{2j+1}\} \tag{27}$$

Therefore, Eq. 2 can be reformulated.

$$\bigwedge_{j=1}^{|T|} \left(\sum_{n_i \in C_{m_j}} e_{n_i,m_j} = 2 \right) \tag{28}$$

IV. Experimental Results

We implemented MuSTNet in Python, which generates CNF files based on our problem formulation and integrates with PaInleSS [22], a parallel SAT solving framework. All experiments were conducted on a machine equipped with two Intel Xeon Gold 6344 processors and 256GB of RAM, where the timeout of the SAT solver is set to 3 hours. Additionally, all generated transistor networks in our experiments were verified via LEC (Logic Equivalence Checking) [23], ensuring functional correctness.

A. Transistor Count Evaluation on Single-Output Logic

We first evaluate our methodology on single-output logic functions, benchmarking against factorization-based, graph-based, and SAT-based approaches using two distinct test cases. The first benchmark comprises 53 handcrafted NSP transistor networks [24], which serve as optimal reference implementations.

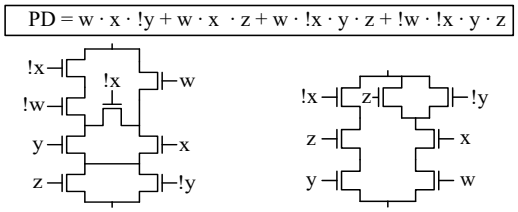
Table II lists down the comparison result of the transistor counts (#T) exclusively for the pull-down networks, following the convention of prior work. The factorization-based method [1], [2], constrained to serial-parallel network configurations, fails to achieve optimality. In contrast, the graph-based approach [6], which accounts for NSP structures, attains near-optimal performance, requiring only three additional transistors. Notably, since all 53 functions are single-stage

TABLE II
TRANSISTOR COUNT EVALUATION FOR 53 HANDMADE NETWORKS [24]

	Optimal	Factor-based		Graph-based			SAT-based	
Ref.	[24]	[2]	[1]	[4]	[5]	[6]	MiniTNtk [10]	MuSTNet
#T	**356**	487	503	516	543	359	**356**	**356**

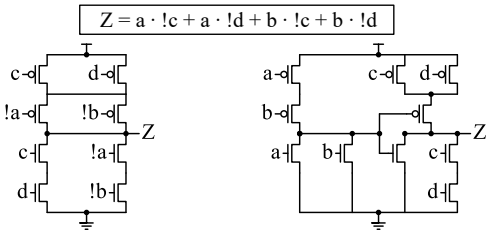
TABLE III
TRANSISTOR COUNT EVALUATION FOR 4-INPUT P-CLASS FUNCTIONS

	Factor-based		Graph-based			SAT-based	
Ref.	[2]	[1]	[4]	[5]	[6]	MiniTNtk [10]	MuSTNet
#T	102668	103049	96804	97174	95595	93031	**76248**



(a) 11T PD network [6] (b) 9T PD network

Fig. 6. Case study on the 4-input P-class: (a) Solution provided by the graph-based method [6] used 11 transistors. (b) Solution generated by our method used only 9 transistors.



(a) 1-stage 12T design [10] (b) 2-stage 10T design

Fig. 7. Case study on the 4-input P-class: (a) Solution generated based on MiniTNtk [10] used 12 transistors. (b) Solution generated by our method used only 10 transistors.

designs, both the SAT-based exact synthesis framework MiniTNtk [10] and our method, MuSTNet, attain the optimal transistor count.

We further extend our evaluation to 3,984 distinct 4-input P-class Boolean functions, generated via input permutations. The results, summarized in Table III, include transistor counts for both PU/PD networks and necessary input inverters. Consistent with prior observations, graph-based and SAT-based methods outperform factorization-based approaches, with MiniTNtk [10] exhibiting a marginal (2.68%) improvement over graph-based optimization [6]. However, both remain focus to single-stage designs, limiting their optimization potential. In contrast, our multi-stage approach achieves significant reductions of 20.2% and 18.0% compared to [6] and [10], respectively. Below, we present two illustrative cases to demonstrate the advantages of our approach.

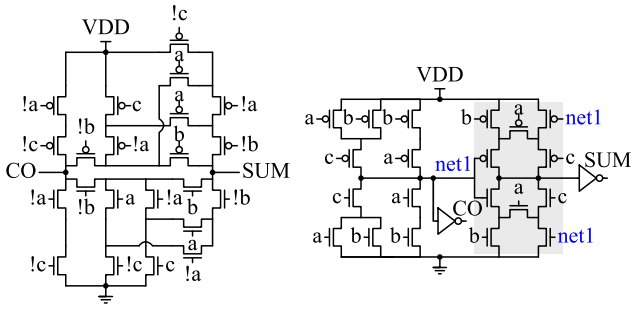
Case study 1: We compare the solution from [6] with our implementation for the function depicted in Fig. 6. Note that in this case, we focus on the PD network, as the reference work omits the PU topology. MuSTNet reduces the transistor count from 11 to 9 (18.2% reduction). Although both designs are 1-stage, further simplification is attainable—highlighting our method's capacity to implicitly optimize the logic of the function for greater efficiency.

Case study 2: A comparison between MiniTNtk [10] and our method is depicted in Fig. 7. It reveals that MiniTNtk employs 12 transistors in a 1-stage design, whereas our 2-stage implementation achieves optimality with just 10 transistors (16.7% reduction). This highlight the superiority of multi-stage optimization in transistor network minimization.

TABLE IV
TRANSISTOR COUNT EVALUATION FOR MULTI-OUTPUT CASES [7].

Functions [7]	#in	#out	MOTO-X [7]	Hybrid [13]	MuSTNet*	Impr.(%)	
						vs. [7]	vs. [13]
BAR1	3	3	25	27	**24**	4.0	11.1
BAR2	4	3	35	36	**30**	14.3	16.7
AGR1	4	3	34	30	**28**	17.6	6.7
AGR2	4	2	28	28	**26**	7.1	7.1
AGR3	4	2	37	42	**33**	10.8	21.4
GUR1	4	3	37	47	**32**	13.5	31.9
GUR2	4	2	22	21	**20**	9.1	4.8
SPEC	4	4	**44**	55	**44**	0.0	20.0
STE1	3	2	24	24	**22**	8.3	8.3
STE2	4	2	**28**	33	**28**	0.0	15.2
STE3	3	3	28	**23**	**23**	17.9	0.0
FADD	3	2	28	27	**24**	14.3	11.1
Avg	-	-	-	-	-	9.7	12.9

*The dummy devices are excluded



(a) 1-stage 28T FADD [7] (b) multi-stage 24T FADD

Fig. 8. Case study on Full-Adder (FADD): (a) Solution provided by MOTO-X [7] used 28 transistors. (b) Solution generated by MuSTNet used only 24 transistors.

B. Transistor Count Evaluation on Multiple-Output Logic

We next evaluate transistor counts in multiple-output complex gates, benchmarking against graph-based [7] and SAT-based method [13] using the test cases in the MOTO-X [7]. Table IV summarizes the results, where #T denotes total transistors including PU/PD networks and input inverters. MuSTNet consistently outperforms prior techniques, achieving reductions of 9.7% and 12.9%, respectively. This improvement, again, is primarily enabled by our multi-stage design, which transcends the 1-stage limitations of existing approaches.

Case study 3: A notable example is the full adder (FADD), shown in Fig. 8. While MOTO-X [7] provides a 1-stage 28T solution (Fig. 8(a)), our multi-stage design achieves a 24T implementation (Fig. 8(b)), reducing transistor count by 14%. Notably, the gray-highlighted region in Fig. 8 (b) demonstrates that our framework not only enables multi-stage hierarchy but also implicitly exploits NSP structures for further optimization.

C. Placement-aware and Runtime Evaluation on Industrial Design

Beyond a mere comparison of transistor counts, we also evaluate the physical area of the generated transistor networks. Using an industrial 4nm technology node, we compare the industrial designs with our two approaches, one incorporating placement constraints

TABLE V
COMPARISON OF PLACEMENT-AWARE AND NON-PLACEMENT-AWARE APPROACHES IN INDUSTRIAL DESIGN

Cell type	#Cells	industrial design (Ind.)		non place-aware		place-aware (PA)		Impr.(%) PA vs. Ind.	
		#T	#CPP	#T	#CPP	#T	#CPP	#T	#CPP
AND/NAND	10	82	54	78	**49**	78	**49**	4.9	9.3
AO/AOI	16	154	102	154	100	154	**95**	0.0	6.9
HA/FA	4	66	38	58	46	60	**36**	9.1	5.3
INV/BUF	2	6	**5**	6	**5**	6	**5**	0.0	0.0
MUX/NMUX	4	70	42	60	43	60	**37**	14.3	11.9
OA/OAI	14	130	88	130	85	130	**80**	0.0	9.1
OR/NOR	11	88	59	84	**53**	84	**53**	4.6	10.2
XOR/XNOR	6	124	72	102	66	106	**64**	14.5	11.1
Total	67	720	460	672	447	678	**419**	5.8	8.9

TABLE VI
BENCHMARKING COMPLEX CELL GENERATION ON 10 CELL PATTERNS

Function	Cell-based		Pimp-only		MuSTNet+Pimp	
	#T	#CPP	#T	#CPP	#T	#CPP
AO22 + AOI211	18	12	18	11	14	**9**
AO22 + NOR4	18	12	18	11	14	**8**
AOI21 + NAND2 + NAND2	14	10	14	9	12	**7**
AOI22 + NAND2 + NAND2	16	11	16	10	14	**9**
NAND2 + NAND2 + NOR3	14	10	14	9	12	**7**
NAND2 + OAI211	12	8	12	8	12	**7**
XNOR2 + NAND4	18	11	18	11	16	**9**
XOR2 + NOR4	18	11	18	11	16	**10**
NAND2 + NAND3	10	7	10	**6**	10	**6**
NOR2 + NOR2	8	6	8	**5**	8	**5**
Total	146	98	146	91	128	**77**

(*place-aware*) and the other disregarding them (*non place-aware*). Table V listed the results where #T is total transistor count and #CPP is contacted poly pitch which reflects the physical area. We acknowledge the author [25] for providing source code for layout generation on 4nm process, ensuring generated layouts are both LVS and DRC clean.

Compared to the industrial design, the placement-aware approach reduces #T by 5.8% while simultaneously reducing #CPP by 8.9%. This improvement mainly comes from optimized transistor stack reordering in OAI/AOI-type cells. As shown in Table V, even with identical #T, #CPP decreases by 6.9% and 9.1% for AOI and OAI cells, respectively. An example of transistor stack reordering is illustrated in Fig. 2 (b) and (d). Moreover, some cells show a greater reduction in #T than in #CPP. This occurs because these cells adopt the NSP structure, which aggressively minimizes transistor count but inherently introduces more odd points in the Euler graph, reducing area efficient. An example of this trade-off is shown in Fig. 2(c).

We then compare the non-placement-aware approach. Interestingly, this method results in the fewest transistors, achieving 6.7% and 0.9% improvements over the industrial design and placement-aware approach, respectively. However, the #CPP shows only a 2.8% reduction compared to the industrial design and even a 6.3% increase compare to the placement-aware approach. This demonstrates the importance of considering transistor placement during transistor network minimization to achieve area-efficient results.

Finally, we compare the runtime improvements achieved with and without the application of acceleration techniques. Figure 9 presents the normalized runtime comparison for each cell type. The results indicate that, by incorporating acceleration constraints, the runtime speedup ranges from 1.3× to 24.9×. It demonstrates the effectiveness of our proposed acceleration techniques. Note that, even for the cell type, XOR/XNOR, the average runtime after applying the acceleration technique remains under one hour, further underscoring the efficiency of our method.

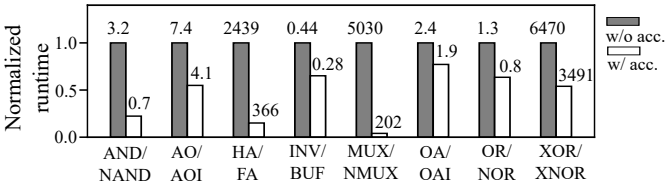


Fig. 9. Runtime (second) comparison w/ and w/o acceleration.

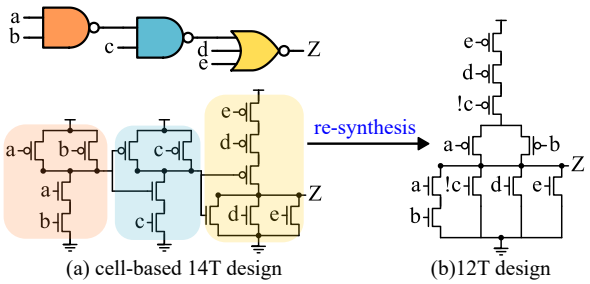


Fig. 10. Case study on complex cell (NAND2+NAND2+NOR3): (a) Cell-based design used 14 transistors with 10 #CPP (b) Our approach employed 12 transistors with only 7 #CPP.

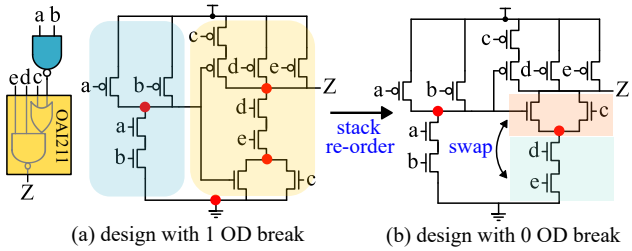


Fig. 11. Case study on complex Cell (NAND2+OAI211): (a) Pimpy-only approach with one OD break requires 8 #CPP. (b) Our approach, with no OD break, requires only 7 #CPP.

D. Practical Application of Complex Gate Generation

To illustrate the practical application of complex gate generation, we extracted 10 frequently occurring cell patterns from an industrial design, as detailed in Table VI. In this experiment, we compare three approaches: (1) traditional cell-based approach (*Cell-based*): #CPP is calculated as the sum of individual cell's #CPP. (2) Physical re-implementation only (*Pimp-only*): we retain the transistor network of each cell but re-implements the physical layout for entire complex cell. (3) MuSTNet with physical implementation (*MuSTNet+Pimp*): the transistor network is re-synthesized by MuSTNet based on the function, followed by physical re-implementation. The result show that *Pimp-only* approach reduces #CPP by 7.1% compare to the cell-based design. This improvement comes from the physical re-implementation enables diffusion sharing at cell boundaries, reducing OD breaks and thus decreasing area. However, the *MuSTNet+Pimp* approach achieves superior optimization by re-synthesizing the transistor network, reducing the transistor count by 12.3% and #CPP by 21.4%. Two examples are illustrated in Fig. 10 and 11.

Case study 4: In Fig. 10, the complex gate's function is optimized, resulting in a single-stage transistor network that achieves a reduction of 14.3% transistor count and 30% #CPP. Although the stack

increases by one level, potentially increasing the delay compared to the original circuit, it remains a viable option for low-cost or power-hungry applications.

Case study 5: In Fig. 11, the overall architecture remains unchanged, but placement constraints implicitly optimize transistor stack ordering, leading to a 12.5% reduction on area (similar to the OAI/AOI cases in prior experiments).

In summary, this experiment demonstrates that our approach enables logic and topology optimization of complex gates, highlighting its potential for advanced IC design.

V. Conclusions And Discussion

In this work, we present MuSTNet, a SAT-based exact synthesis framework for multi-stage transistor networks with integrated placement awareness. Our approach introduces two key innovations: (1) Multi-stage transistor network synthesis: We formulate the synthesis problem as a SAT problem, enabling multi-stage optimization by allowing internal nodes to serve as the 'gate' for other transistors. (2) Placement-aware transistor synthesis: We incorporate placement constraints during network minimization to ensure area efficiency. Experimental results demonstrate that MuSTNet outperforms existing state-of-the-art methods, offering a promising solution for further optimizations in IC design.

While SAT-based exact synthesis guarantees optimality, it currently struggles with scalability when optimizing large complex gates (e.g., multipliers). To address this limitation, future work will explore enhanced techniques (e.g., topology families techniques [16]) or hybrid approaches (e.g., guided exact synthesis [13]), aiming to balance optimality with computational efficiency.

Acknowledgment

This research work is supported in part by the MediaTek Advanced Research Center (MARC).

References

[1] L. S. da Rosa, A. I. Reis, R. P. Ribas, F. d. S. Marques, and F. R. Schneider, "A comparative study of cmos gates with minimum transistor stacks," in *Proceedings of the 20th Annual Conference on Integrated Circuits and Systems Design*, ser. SBCCI '07. New York, NY, USA: Association for Computing Machinery, 2007, p. 93–98. [Online]. Available: https://doi.org/10.1145/1284480.1284511

[2] M. G. A. Martins, L. Rosa, A. B. Rasmussen, R. P. Ribas, and A. I. Reis, "Boolean factoring with multi-objective goals," in *2010 IEEE International Conference on Computer Design*, 2010, pp. 229–234.

[3] R. Reis, "Design automation of transistor networks, a new challenge," in *2011 IEEE International Symposium of Circuits and Systems (ISCAS)*, 2011, pp. 2485–2488.

[4] V. N. Possani, R. S. de Souza, J. S. Domingues, L. V. Agostini, F. S. Marques, and L. S. da Rosa, "Optimizing transistor networks using a graph-based technique," *Analog Integrated Circuits and Signal Processing*, vol. 73, pp. 841–850, 2012.

[5] D. Kagaris and T. Haniotakis, "A methodology for transistor-efficient supergate design," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 15, no. 4, pp. 488–492, 2007.

[6] V. N. Possani, V. Callegaro, A. I. Reis, R. P. Ribas, F. de Souza Marques, and L. S. da Rosa, "Graph-based transistor network generation method for supergate design," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 24, no. 2, pp. 692–705, 2016.

[7] D. Kagaris, "Moto-x: A multiple-output transistor-level synthesis cad tool," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 35, no. 1, pp. 114–127, 2016.

[8] M. S. Cardoso, G. H. Smaniotto, A. A. O. Bubolz, M. T. Moreira, L. S. da Rosa, and F. d. S. Marques, "Libra: An automatic design methodology for cmos complex gates," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 65, no. 10, pp. 1345–1349, 2018.

[9] C.-P. Lu, I. H.-R. Jiang, and C.-W. Yang, "Late breaking results: Design dependent mega cell methodology for area and power optimization," in *2020 57th ACM/IEEE Design Automation Conference (DAC)*, 2020, pp. 1–2.

[10] W. Xiao, S. Han, Y. Yang, S. Yang, C. Zheng, J. Chen, T. Liang, L. Li, and W. Qian, "Minitntk: An exact synthesis-based method for minimizing transistor network," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2023, pp. 01–09.

[11] C.-H. Hsu, X. Xu, H. Chen, D. Ruic, and D. Z. Pan, "Transplace: A scalable transistor-level placer for vlsi beyond standard-cell-based design," in *Proceedings of the 29th Asia and South Pacific Design Automation Conference*, ser. ASPDAC '24. IEEE Press, 2024, p. 312–318. [Online]. Available: https://doi.org/10.1109/ASP-DAC58780.2024.10473978

[12] C.-K. Cheng, B. Kang, B. Lin, and Y. Wang, "Standard cell layout generation: Review, challenges, and future works," in *Proceedings of the 30th Asia and South Pacific Design Automation Conference*, ser. ASPDAC '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 372–378. [Online]. Available: https://doi.org/10.1145/3658617.3703146

[13] L. Feng, R. Liang, and H. Kong, "Hybrid exact and heuristic efficient transistor network optimization for multi-output logic," in *2025 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2025, pp. 1–6.

[14] H. Kessler, M. Porto, L. Da Rosa, and V. V. Camargo, "Standard cell and supergates designs: An electrical comparison on 4-input logic functions," in *2022 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2022, pp. 1744–1748.

[15] V. Callegaro, F. d. S. Marques, C. E. Klock, L. S. da Rosa, R. P. Ribas, and A. I. Reis, "Switchcraft: a framework for transistor network design," in *Proceedings of the 23rd Symposium on Integrated Circuits and System Design*, ser. SBCCI '10. New York, NY, USA: Association for Computing Machinery, 2010, p. 49–53. [Online]. Available: https://doi.org/10.1145/1854153.1854167

[16] W. Haaswijk, M. Soeken, A. Mishchenko, and G. De Micheli, "Sat-based exact synthesis: Encodings, topology families, and parallelism," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 4, pp. 871–884, 2020.

[17] H. Riener, W. Haaswijk, A. Mishchenko, G. De Micheli, and M. Soeken, "On-the-fly and dag-aware: Rewriting boolean networks with exact synthesis," in *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2019, pp. 1649–1654.

[18] Uehara and Vancleemput, "Optimal layout of cmos functional arrays," *IEEE Transactions on Computers*, vol. C-30, no. 5, pp. 305–312, 1981.

[19] D. Radhakrishnan, S. Whitaker, and G. Maki, "Formal design procedures for pass transistor switching circuits," *IEEE Journal of Solid-State Circuits*, vol. 20, no. 2, pp. 531–536, 1985.

[20] R. Carden, J. Li, and C.-K. Cheng, "A global router with a theoretical bound on the optimal solution," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 15, no. 2, pp. 208–216, 1996.

[21] D. Park, D. Lee, I. Kang, C. Holtz, S. Gao, B. Lin, and C.-K. Cheng, "Grid-based framework for routability analysis and diagnosis with conditional design rules," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 12, pp. 5097–5110, 2020.

[22] L. Le Frioux, S. Baarir, J. Sopena, and F. Kordon, "Painless: a framework for parallel sat solving," in *International Conference on Theory and Applications of Satisfiability Testing*. Springer, 2017, pp. 233–250.

[23] Cadence Design Systems, Inc, "Conformal equivalence checker datasheet — cadence," 2025. [Online]. Available: https://www.cadence.com/en_US/home/resources/datasheets/conformal-equivalence-checker-ds.html

[24] L. L. Federal Univ. Rio Grande do Sul. (Oct. 2012) Catalog of 53 handmade optimum switch networks. [Online]. Available: http://www.inf.ufrgs.br/logics/docman/53_NSP_Catalog.pdf

[25] J.-C. Tsai, W.-M. Hsu, Y.-T. Hsieh, Y.-J. Li, W. Huang, C. N. Ho, H.-M. Huang, J.-H. Yang, H.-L. Huang, A. C. W. Liang, and C. H. P. Wen, "Maxcell: Ppa-directed multi-height cell layout routing optimization using anytime maxsat with constraint learning," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, ser. ICCAD '24. New York, NY, USA: Association for Computing Machinery, 2025. [Online]. Available: https://doi.org/10.1145/3676536.3676706